Amb la presentació d'aquest document els autors afirmen a tots els efectes que tant la pràctica informada com el propi informe és exclusivament fruit del seu treball personal i en cap cas s'ha plagiat total o parcialment.

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Escola Politècnica Superior d'Enginyeria de Manresa

Weather station system

Activity 2: LoRa-based communication with MQTT integration — Internet of Things Connectivity

Aarón Alcalá Marcos García Jaume Pulido Mikael Rodríguez

October 14, 2026

Contents

1 Introduction **2**

2 LoRa Configuration **2**
2.1 Radio Parameters . . . 2
2.2 Compatibility, Interference and Software Design . . . 3

3 TX node (`src/tx/main.cpp`) **3**
3.1 Configuration . . . 3
3.2 Initialisation . . . 4
3.3 Main loop . . . 4

4 RX node **5**
4.1 Main program (`main.cpp`) . . . 5
4.2 MQTT . . . 7
4.3 Frontend . . . 7

5 Experiments **7**
5.1 Transmission power vs. energy consumption and signal strength . . . 7
5.1.1 Test procedure . . . 7
5.1.2 Expected behaviour . . . 8
5.1.3 Results . . . 9
5.1.4 Discussion . . . 9
5.2 Bandwidth and data rate . . . 10
5.3 Quality of Service: packet loss vs. distance . . . 11
5.4 Discussion of the results . . . 11

6 Answers to Guiding Questions **11**

7 Conclusions **11**

1 Introduction

This report documents Activity 2 of the Weather station system, which extends the MQTT chain of Activity 1 by introducing a LoRa link between two ESP32 boards: a transmitter (TX) node that reads the DHT11 sensor and sends the data over LoRa, and a receiver (RX) node that forwards what it receives to the Mosquitto broker over MQTT, so that the dashboard remains unchanged.

For this activity, two groups joined forces: Aarón Alcalá and Marcos García on one side, and Jaume Pulido and Mikael Rodríguez on the other. The work was split so that each group programmed one of the two nodes. Jaume and Mikael's group developed the TX node, including the handling of the DHT11 sensor, and Aarón and Marcos's group developed the RX node, where the WiFi connection, the MQTT publication to the Mosquitto broker and the web frontend keep the same idea and structure as in Activity 1.

2 LoRa Configuration

The LoRa communication system enables two ESP32-based boards to exchange sensor information wirelessly. The objective is to provide a long-range communication link suitable for IoT applications while keeping the implementation modular and easy to maintain.

The configuration and communication logic are separated into different files. The `LoRaConfig.h` file defines the radio configuration structure, while `config.h` contains the default radio parameters and hardware pin assignments. The `LoRa_IOT.h` and `LoRa_IOT.cpp` files define the communication interface and its implementation. This separation allows the radio parameters to be modified without changing the application logic.

2.1 Radio Parameters

The radio configuration is centralised in `config.h` and passed to the initialisation function through the `LoRaConfig` structure. The main parameters and their design rationale are described below.

- **Frequency (868 MHz):** Selected for operation in the European 868 MHz band, subject to applicable regulations. Both boards must use the same frequency to communicate.
- **Bandwidth (250 kHz):** Provides a relatively high data rate compared with narrower bandwidths, making it suitable for transmitting small sensor messages periodically.
- **Spreading factor (SF7):** Selected to favour a higher data rate and shorter transmission times. This is suitable for small sensor messages, provided that the required communication range and signal quality are maintained.
- **Coding rate (4/5):** Adds forward error correction to improve resilience against transmission errors while limiting additional transmission overhead.
- **Sync word (0x12):** Helps distinguish our LoRa network from networks using different sync words. Both boards must use the same value.
- **Preamble length (8 symbols):** Supports packet detection and synchronisation while limiting transmission overhead.

- **CRC enabled:** Allows the receiver to detect corrupted packets and identify transmission errors.

The transmit power is also configurable. Its final value must be compatible with the radio module and comply with the applicable regional regulations.

2.2 Compatibility, Interference and Software Design

Reliable communication requires both boards to use the same radio parameters (frequency, bandwidth, spreading factor, coding rate and sync word), which is why they are centralised in a single configuration shared by the TX and RX nodes. The sync word reduces the chance of accepting packets from unrelated LoRa networks, but the radio channel is shared, so interference and collisions caused by other devices can still happen. It also provides neither encryption nor sender authentication.

On the software side, the design is modular: the radio configuration is kept apart from the application logic, and the public interface offers initialisation, transmission and reception functions built on top of the Arduino LoRa library. A GPIO interrupt is also configured on the radio so that the arrival of a packet can be signalled to the main program without constant polling (see Section 4). The final parameters should be validated at the expected operating distances and environmental conditions (see Section 5).

3 TX node (`src/tx/main.cpp`)

The transmitter is the node placed at the weather station. Its only task is to read the DHT11 sensor periodically and send each reading over LoRa to the receiver, which acts as a gateway towards the MQTT broker. Because it does not need any Internet connectivity, the transmitter does not use WiFi or MQTT at all, which keeps the firmware small and reduces its power consumption.

The program is built with PlatformIO in its own environment (`env:tx`), which compiles only the `src/tx/` folder through `build_src_filter`. The radio code is shared with the receiver and lives in the `lib/lora` library (`LoRa_IOT.h/.cpp`), so the main file only contains the application logic.

3.1 Configuration

The radio parameters of the transmitter are the ones described in Section 2.1, loaded from `include/config.h` into the same `LoRaConfig` structure used by the receiver. The parameters that are specific to the transmitter are summarised in Table 1.

Parameter	Value
Sensor	DHT11 on GPIO 13
Send interval	5 s (`LORA_SEND_INTERVAL_MS`)

Table 1: Application parameters specific to the transmitter.

3.2 Initialisation

The `setup()` function opens the serial port for debugging, starts the DHT11 driver and initialises the radio with `LoRaIOT_setup()`:

```
DHT dht(DHT_PIN, DHT_TYPE);
LoRaConfig loraConfig; // default values from config.h
int counter = 0; // packet id

void setup(){
    Serial.begin(115200);
    dht.begin();

    if (!LoRaIOT_setup(loraConfig))
        while (1); // stop if the radio does not answer
}
```

Listing 1: Initialisation of the transmitter.

`LoRaIOT_setup()` starts the SPI bus on the pins of the on-board SX1276, sets the chip-select, reset and DIO0 pins, calls `LoRa.begin()` with the carrier frequency and then applies the spreading factor, bandwidth, coding rate, preamble length, sync word and transmission power, and enables the CRC. If the radio does not respond, the function returns `false` and the program stops, since the node is useless without the radio.

3.3 Main loop

Every iteration of `loop()` performs three steps: it reads the sensor, builds the message and sends it.

```
void loop(){
    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();

    if (isnan(temperature) || isnan(humidity)) {
        Serial.println("Error leyendo el DHT11");
        delay(2000);
        return; // retry without sending anything
    }

    String message = "{\"id\":" + String(counter) + ",\"temperature\":" + String(temperature,
        ↪  1) + ",\"humidity\":" + String(humidity, 1) + "}";

    if (!LoRaIOT_send(message))
        Serial.println("Error enviando el paquete");

    counter++;
    delay(LORA_SEND_INTERVAL_MS);
}
```

Listing 2: Main loop of the transmitter.

Sensor reading. The DHT11 returns `NaN` when a reading fails (for example, if it is read too often or a wire is loose). In that case the iteration is discarded and repeated after 2 s, so invalid values are never transmitted.

Message format. Each reading is encoded as a small JSON object:

```
{"id":12,"temperature":23.4,"humidity":55.0}
```

Values are rounded to one decimal, which matches the resolution of the DHT11. JSON was chosen because the receiver can parse it directly with ArduinoJson and the format is easy to read in the serial monitor. The field `id` is a counter that increases with every packet; the receiver can use it to detect lost packets by looking for gaps in the sequence. The message is about 43–45 bytes long, far below the 255-byte limit of a LoRa packet.

Transmission. `LoRaIOT_send()` wraps the three calls of the Arduino LoRa library: `beginPacket()`, `print()` and `endPacket()`. The last one is blocking: it returns when the radio signals the end of the transmission (*TxDone*), so when `LoRaIOT_send()` returns the packet has already been sent.

Send period. After each packet the node waits `LORA_SEND_INTERVAL_MS` (5 s). This respects the DHT11 minimum sampling period (1 s) and keeps the channel occupancy low: with a time on air of around 44 ms (see Section 5.1.2), the duty cycle is below 1%, which is the limit of the EU868 sub-band used.

4 RX node

The RX node is the bridge between the LoRa link and the MQTT chain built in Activity 1. Its task is to wait for the packets sent by the TX node, extract the temperature and humidity from them and republish them to the Mosquitto broker, so that the dashboard keeps working exactly as before. The WiFi connection, the MQTT publication and the web frontend follow the same idea and structure as in Activity 1, so the only new element on this node is the LoRa reception.

4.1 Main program (`main.cpp`)

The program is built with PlatformIO in its own environment (`env:rx`), which compiles only the `src/rx/` folder through `build_src_filter`.

The `main.cpp` of the RX node reuses the `WiFiManager` and `MqttPublisher` classes of Activity 1 and adds the LoRa functions provided by the communication library described in Section 2. Besides the WiFi and MQTT objects, the program only needs one extra global variable, `rxFlag`, a `volatile bool` that signals that a LoRa packet has arrived.

Initialization. In `setup()`, the WiFi manager is started as in Activity 1, and the radio is configured through `LoRaIOT_setup(LoRaConfig())`. This is the same call used by the TX node, so both boards share exactly the same radio configuration (frequency, spreading factor, bandwidth, coding rate, preamble length and sync word); otherwise the receiver would not be able to decode the packets. If the radio fails to start, the program stops, since the node would be useless without it. Finally, `LoRa.receive()` is called to put the radio in continuous receive mode. This call is essential: after `LoRa.begin()` the module is not listening by default, so without it no packet would ever be detected and the interrupt described below would never fire.

Interrupt-driven reception. Instead of repeatedly asking the radio whether a packet has arrived, the RX node relies on a hardware interrupt. An interrupt makes the microcontroller run a function only when a given event happens, which here is the arrival of a packet. During `LoRaIOT_setup()`, `attachInterrupt()` associates the DIO0 pin of the LoRa module with the function `onLoRaRxIRQ()`, which is executed on every rising edge of that pin, i.e. when the module signals that a complete packet has been received:

```
extern volatile bool rxFlag;

void IRAM_ATTR onLoRaRxIRQ()
{
    // The interrupt only signals that a packet has arrived
    rxFlag = true;
}
```

The interrupt routine does nothing but raise the flag, since this kind of routine must be as short as possible. The variable is declared `volatile` in `main.cpp` because it is modified from the interrupt and read from `loop()`, so the compiler must not assume that its value stays unchanged between reads. The library file accesses it through `extern`, and the `IRAM_ATTR` attribute keeps the routine in the ESP32 internal RAM so it can run quickly. The heavy work (reading the packet, parsing it and publishing it) is done later in the main loop, outside the interrupt.

Main loop. On every iteration, `loop()` follows these steps:

1. The WiFi manager is updated, and the iteration is skipped if there is no connection yet.
2. If the MQTT connection has not been established, it is attempted again, and `mqttPublisher.loop()` is called to keep it alive. These steps work exactly as in Activity 1.
3. If `rxFlag` is set, it is cleared first and then the packet is read with `LoRaIOT_receive()`.
4. If a packet was read correctly and the MQTT connection is available, the message is handed to `publishReading()`.

In contrast with Activity 1, the publication is no longer periodic: nothing is sent to the broker until a LoRa message arrives, so the publishing rate is set by the TX node.

Packet reception. The `LoRaIOT_receive()` function checks for an incoming packet using `LoRa.parsePacket()`. If no packet is available, it returns `false`. Otherwise, it reads the packet byte by byte using `LoRa.available()` and `LoRa.read()`, storing the received data in the `message` string. It then prints the message and its RSSI value, which indicates the received signal strength. Finally, `LoRa.receive()` puts the radio back into reception mode, and the function returns `true` to indicate that a packet has been received.

From the LoRa message to MQTT. The message received over LoRa is a JSON string. `publishReading()` deserializes it with *ArduinoJson*, discarding the packet and reporting the error if the parsing fails. Then it extracts only the temperature and the humidity, ignoring the rest of the fields of the received JSON, such as the packet counter `id`, since that information is not going to be sent over MQTT. Finally, it builds a new JSON with the `temp` and `hum` fields through the `MqttPublisher` and publishes it to the broker:

```
StaticJsonDocument<200> doc;
DeserializationError error = deserializeJson(doc, message);
```

```
3
4  if (error) return false;
5  float temperature = doc["temperature"];
6  float humidity = doc["humidity"];
7
8  publisher.addField("temp", temperature);
9  publisher.addField("hum", humidity);
10 publisher.publish(topic, retained);
```

Rebuilding the JSON with the same fields as in Activity 1 is what allows the rest of the chain to remain unchanged: the broker and the dashboard receive the same message on `sensor/1` as if the sensor were connected directly to the node.

4.2 MQTT

The MQTT part of the RX node is the same as in Activity 1. The `WiFiManager` state machine handles the connection to the WiFi network without blocking, and the `MqttPublisher` class connects to the Mosquitto broker, reconnects automatically if the connection is lost, and serializes and publishes the JSON message. The only difference is that the readings now come from the LoRa packets instead of a local sensor.

4.3 Frontend

The frontend does not change either. `dashboard.py` keeps subscribing to `sensor/+`, storing the readings in bounded queues and drawing the live charts with Dash and Plotly, as described in Activity 1. Since the RX node publishes the same JSON format and topic, the dashboard cannot tell whether the data comes from a sensor wired to the node or from a remote one over LoRa.

5 Experiments

5.1 Transmission power vs. energy consumption and signal strength

The aim of this experiment is to evaluate how the transmission power affects the signal strength at the receiver and the energy that the transmitter needs to send each packet. In a battery-powered sensor node there is a trade-off between both: a higher power gives a stronger and more robust link, but drains the battery faster.

5.1.1 Test procedure

The experiment is automated with a PlatformIO test, `test/test_tx_power/test_tx_power.cpp`, written with the Unity framework. It runs on the transmitter board and is launched with its own environment, `env:tx_power`, which extends `env:tx` and only runs this test:

```
pio test -e tx_power
```

The test performs the following steps:

1. It initialises the radio with `LoRaIOT_setup()` and the same configuration as the normal firmware (SF 7, 250 kHz bandwidth, coding rate 4/5, 868 MHz). If this fails, the test fails.
2. For each power level in $\{2, 5, 8, 11, 14, 17, 20\}$ dBm, it sets the power with `LoRa.setTxPower()` and sends 10 packets, 500 ms apart. Each packet carries the power used and a sequence number, so the receiver can tell which level it belongs to:

```
{"test":"tx_power","power":14,"seq":3}
```

3. For each packet it measures the time taken by `LoRaIOT_send()`. Since `endPacket()` blocks until *TxDone*, this is the time on air of the packet plus a small SPI overhead.

4. At the end of each level it prints a CSV line with the power, the number of packets sent and failed, and the average time on air, and waits 3 s before the next level.

5. Finally, it restores the default power from `config.h`. The test passes if every packet was transmitted successfully.

The sweep was repeated continuously, so several sweeps were recorded in a single run. The receiver runs its normal firmware (`env:rx`) without any change. For every packet it prints the payload and its RSSI on the serial monitor, so the RSSI of each power level is obtained from the receiver's log.

The power consumption cannot be measured from the firmware, so the transmitter was powered from a laboratory power supply at 5 V, and the current shown by the supply was recorded on video during the sweep. The reading alternates between a low value while the node is idle between packets and a higher value while it transmits. The energy spent per packet is then

$$E_{\text{packet}} = V \cdot I \cdot t_{\text{air}}, \tag{1}$$

where $V = 5$ V is the supply voltage, I the current during the transmission and t_{air} the time on air reported by the test. Two values are given: the energy of the whole board, using the current during the transmission, and the energy of the radio alone, using the increase over the idle current, $\Delta I = I_{\text{TX}} - I_{\text{idle}}$.

Setup. The two Heltec WiFi LoRa 32 V2 boards were placed on a table in the laboratory, 1 m apart, with line of sight and their antennas connected.

5.1.2 Expected behaviour

The time on air of a LoRa packet only depends on the spreading factor, the bandwidth, the coding rate, the preamble and the payload length, not on the transmission power. With the parameters used (SF 7, 250 kHz bandwidth, coding rate 4/5 and an 8-symbol preamble), the symbol time is

$$T_{\text{sym}} = \frac{2^{SF}}{BW} = \frac{2^7}{250\,\text{kHz}} = 0.512 \text{ ms}, \tag{2}$$

and, using the formula from the SX1276 datasheet, the 37–38-byte test packets need 68 payload symbols. Adding the preamble ($8 + 4.25$ symbols), the expected time on air is about

$$t_{\text{air}} = (12.25 + 68) \cdot 0.512 \text{ ms} \approx 41 \text{ ms}. \tag{3}$$

For the 43-byte weather messages of the normal firmware it is about 44 ms. Therefore, the measured time on air should be the same at every power level, and the energy per packet should change only because of the current drawn by the power amplifier. The RSSI is expected to grow roughly 1 dB for every dB of extra transmission power, until the receiver saturates when the boards are very close.

5.1.3 Results

The time on air measured by the transmitter was 42.0–42.1 ms for every packet, at every power level. Five sweeps were recorded; the receiver was restarted once during the second sweep, so the first and second sweeps are incomplete. In total, 230 valid packets were received. Table 2 gives the RSSI statistics of all the packets received at each level, and Table 3 the current and energy per packet. The idle current of the board was 42 mA.

Power (dBm)	**Packets**	**Mean RSSI** (dBm)	**Std. dev.** (dB)	**Min** (dBm)	**Max** (dBm)
2	30	−47.4	2.6	−53	−43
5	40	−44.8	1.9	−49	−42
8	29	−41.3	2.6	−45	−37
11	37	−39.2	4.4	−49	−34
14	30	−35.9	2.6	−41	−33
17	29	−33.0	3.0	−39	−29
20	30	−30.9	2.4	−35	−29

Table 2: RSSI at the receiver for each transmission power (all sweeps, only levels with at least 8 packets received).

Power (dBm)	I_{TX} (mA)	ΔI (mA)	t_{air} (ms)	**Energy (board)** (mJ)	**Energy (radio)** (mJ)
2	50	8	42.1	10.53	1.68
5	51	9	42.1	10.74	1.89
8	52	10	42.1	10.95	2.11
11	54	12	42.1	11.37	2.53
14	57	15	42.1	12.00	3.16
17	61	19	42.1	12.84	4.00
20	65	23	42.1	13.68	4.84

Table 3: Current and energy per packet at 5 V ($I_{\mathrm{idle}} = 42$ mA).

5.1.4 Discussion

Time on air. The measured time on air was 42.0–42.1 ms at every power level, as expected, since it does not depend on the transmission power. It is about 1 ms longer than the theoretical 41 ms of Section 5.1.2. This difference comes from the measurement method: the time includes writing the payload to the radio over SPI and the delay until the *TxDone* interrupt is detected.

Signal strength. The RSSI grows almost linearly with the transmission power: a linear fit gives a slope of 0.93 dB per dB, very close to the ideal 1 dB per dB. Going from 2 to 20 dBm raised the mean RSSI from −47.4 to −30.9 dBm. The spread between sweeps (standard deviation of 2–4 dB, and up to 10 dB between the means of two sweeps at the same level) is larger than the spread within a sweep. This suggests small changes in the environment during the test, such as people moving or the boards being touched, rather than random noise. No saturation was observed,

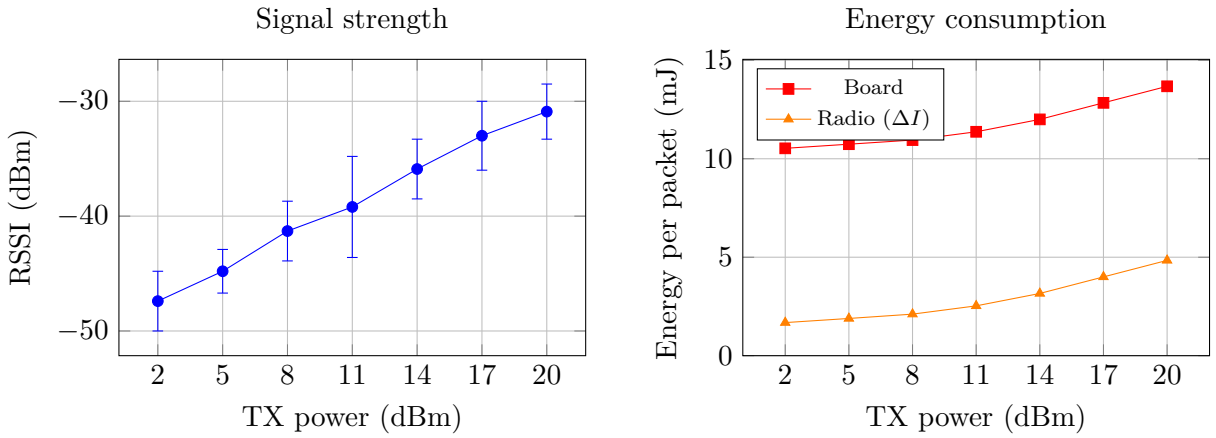


Figure 1: Mean RSSI at the receiver with its standard deviation (left) and energy per packet at the transmitter (right) as a function of the transmission power.

even at 20 dBm. Practically no packets were lost: the only gaps in the sequence numbers are at 8 and 17 dBm, not at the lowest power, and the incomplete first two sweeps are explained by the receiver restart. Even at 2 dBm the RSSI (−47.4 dBm) is far above the sensitivity of the SX1276 for SF 7 and 250 kHz in the 868 MHz band, −120 dBm according to its datasheet (Table 10) [**?**]. This is a link margin of more than 70 dB, so at 1 m every power level gives a reliable link.

Energy consumption. The current during the transmission grows with the power, from 50 mA at 2 dBm to 65 mA at 20 dBm, while the idle current stays at 42 mA. Since the time on air is constant, the energy per packet follows the current: the radio alone needs 1.7 mJ per packet at 2 dBm and 4.8 mJ at 20 dBm, about 2.9 times more. Going from 14 to 20 dBm increases it by 53% (from 3.2 to 4.8 mJ) for only about 5 dB more signal. For the whole board the difference is smaller (10.5 to 13.7 mJ), because most of the energy is consumed by the ESP32, which is active all the time.

These currents should be read as a trend rather than absolute values. A transmission lasts only 42 ms, which is shorter than the refresh time of the power supply display, so the value shown is partly averaged with the idle current. The SX1276 datasheet gives a supply current of 87 mA at 17 dBm and 120 mA at 20 dBm for the radio alone (Table 6) [**?**], clearly higher than the increase measured here (19 and 23 mA). To measure the real peak current, the transmitter should send continuously (no pause between packets), or the current should be measured with an oscilloscope and a shunt resistor.

Trade-off. At the distance used, even the lowest power gives a reliable link with a large margin, so increasing the power only spends more energy without improving the delivery rate. For a battery-powered node, the best choice is the lowest power that keeps a safe margin at the real deployment distance. Moreover, in the EU868 band the legal limit is usually 14 dBm ERP, so 17 and 20 dBm were used only for this laboratory test.

5.2 Bandwidth and data rate

⟨Variación de bandwidth / spreading factor; resultados en tabla o gráfica.⟩

5.3 Quality of Service: packet loss vs. distance

⟨Medida de pérdida de paquetes a distintas distancias; resultados.⟩

5.4 Discussion of the results

⟨Análisis conjunto de los resultados.⟩

6 Answers to Guiding Questions

Q1. How does LoRa compare to Wi-Fi and Bluetooth in terms of range and power consumption?

LoRa offers a much longer range than Wi-Fi and Bluetooth, from hundreds of metres to several kilometres under suitable conditions, while keeping a low average power consumption. The price is a low data rate, so it is only suitable for small messages. Wi-Fi is much faster, but it has a shorter range, needs an access point and generally consumes more energy, while Bluetooth also consumes little but only works at short distances (the exact figures depend on the hardware, the Bluetooth version and the environment). LoRa fits our project because the boards only exchange a small sensor reading every few seconds and no network infrastructure is needed.

Q2. What are the trade-offs of using lower data rates in LoRa transmission?

Lowering the data rate (by increasing the spreading factor or reducing the bandwidth) makes the signal easier for the receiver to detect, which increases the sensitivity and the potential range. The price is a longer time on air: the radio stays on for longer, so each message costs more energy, the shared channel is occupied for longer and the probability of collisions grows, the regulations allow fewer messages per hour and the data arrives with more delay. Therefore, a lower data rate does not necessarily mean a lower consumption. In our case, SF7 with 250 kHz keeps the transmissions short, which is appropriate for small sensor messages as long as the receiver decodes them reliably; a higher spreading factor would only be needed for more range.

Q3. How does increasing transmission power affect battery life and interference?

A higher transmission power improves the received signal strength and can extend the range, but the radio draws more current while transmitting, so the battery lasts less (the consumption does not grow in the same proportion as the radiated power, since it also depends on the efficiency of the radio). It also increases the interference with other devices sharing the same frequencies and, if the boards are very close, it can overload the receiver. The best strategy is therefore to use the minimum power that keeps a reliable link, and the final setting must comply with the radio regulations. In our project, the RSSI measurements and the packet reception of the power experiment (Section 5.1) help to find it.

7 Conclusions

⟨Conclusiones conjuntas.⟩